Building an LLM Python debugger agent with the new Claude 3.5 Sonnet

Building a AI powered coding agent with Claude 3.5 Sonnet!

Brett Young

Created on October 24 | Last edited on October 25

In this project, we're building a command-line interface (CLI) AI-powered Python debugger agent called Code Watchdog. This tool uses the Anthropic's upgraded Claude 3.5 Sonnet API to analyze logs, recent console output, and error traces, generating solutions to coding issues when you explicitly instruct it to do so, mitigating the need for developers to manually gather context and construct complex prompts to solve code issues.

With these sorts of projects, I've found it best to start with a simplified version before adding too many complex features. Building it initially in its "non-agentic" form allows for easier testing and product validation and ensures that the core functionality is solid before going too far into agent territory.

Jump to the tutorial

The new Claude 3.5 Sonnet: An agent API?

Claude 3.5 Sonnet is designed with a focus on coding and workflow automation, setting it apart from general-purpose models like GPT-4. While models such as GPT-4 excel in broad conversational abilities, Claude specializes in agentic tasks, excelling at autonomous problem-solving and code-related workflows. This makes it particularly effective when integrated with our Debugger Agent.

Claude 3.5 Sonnet and Haiku offer significant improvements in coding performance. **Sonnet's SWE-bench score jumped from 33.4% to 49.0%, outperforming OpenAI's o1-preview and excelling in software workflows and planning tasks**. These advancements make Claude models perfect for our code debugger, offering better performance for providing precise fixes, smooth automation, and actionable insights directly within coding environments.

The tool we'll be building, called **Code Watchdog,** isn't like traditional debuggers that pause code with breakpoints. Instead, it automatically monitors file changes, logs activity, and captures errors in the background. When you need it, you can call on the tool to provide AI-powered fixes and insights based on the collected logs. This approach makes it especially useful for troubleshooting errors or bugs that appear after code has been executed.

The tool also integrates with W&B Weave, which is a tool that allows you to visualize and analyze inputs and outputs to functions in your code, helping to quickly identify missing

context or issues with GenAI apps. This tool acts seamless way to monitor production behavior, and even debug code without interrupting your workflow.

A preview of how it works

With Code Watchdog, you no longer need to copy and paste code or error messages into an AI chat interface to get help. Instead, the tool captures the necessary logs and errors for you in real time. When you're ready, you can simply run a short command in your terminal to generate an AI-powered solution.

For example, if you encounter a bug or want to analyze recent changes, all you need to do is run a command like the following (cw is short for Code Watchdog):

```
cw 3 “Please solve the bug”
```

*This command will gather logs from the last **three** modified, created, or executed python files, previous console output and the previous error message, and query Anthropic's Claude 3.5 API to generate a suggested fix.* In just a few seconds, you'll receive actionable insights and code snippets tailored to the issue at hand, **all without leaving your terminal.**

Once the response is received, Code Watchdog doesn't just print the solution to the terminal—it opens the generated fix directly in VSCode for easy visualization and further editing.

Why we are building a coding agent

Debugging is one of the most time-consuming aspects of software development. Developers often spend hours sifting through console logs, error traces, and code changes to understand what went wrong and how to fix it. Traditional debuggers can be useful, but they rely heavily on setting breakpoints and pausing execution, which can interrupt the flow of development. Also, code completion tools like GitHub Copilot assist with writing new code but fall short when it comes to debugging and troubleshooting existing code, leaving developers to track down errors manually.

We are building Code Watchdog to solve this problem by automating the task of providing context to the LLM used for debugging. Instead of forcing developers to manually track changes and errors, Code Watchdog works quietly in the background, capturing file activity and runtime logs. When issues arise, developers can request the tool to generate insights and fixes based on the collected data.

The goal is to automate the debugging process, reduce friction, and eliminate the tedious task of hunting down issues manually. With the integration of tools like Weave for log

analysis and Anthropic's Claude 3.5 API for AI-powered solutions, Code Watchdog ensures that developers can quickly identify and resolve issues without disrupting their workflow. This allows more time to focus on building and less time on firefighting bugs.

TLDR on Code Watchdog's Backend

Now, a core component of this project is a mechanism that captures all relevant code activity—such as file modifications, script executions, console outputs, and error logs—and stores it in a structured way. This enables Code Watchdog to provide meaningful insights when you encounter an issue, ensuring that debugging is seamless and efficient. Rather than requiring developers to manually track or log this information, the backend infrastructure takes care of it automatically, allowing you to focus on writing code.

I won't go into the details of how to implement the backend, but if you're interested, feel free to explore the repo here, which contains all the scripts and files needed to understand how everything works. Additionally, I've provided a simple setup script that will set up everything for you in a new Conda environment, so you can get started using Code Watchdog without any hassle.

I will briefly cover how the backend of code watchdog works to automatically log your activity during programming. Don't worry if you have never heard of some of these logging mechanisms, as the only way you likely would or should know about them is if you have spent time maintaining the conda project, or worked on a similar project.

We achieve monitoring and logging through two key components: sitecustomize.py and Conda activation scripts. The `sitecustomize.py` file acts as a Python hook that runs automatically whenever Python starts, allowing us to log file activity and errors. Additionally, we use scripts in the `activate.d` directory, which are executed when the Conda environment is activated. These scripts start background processes that monitor file changes in real time. This combination ensures that every file change or execution is captured, with logs generated automatically, without requiring manual intervention from the user.

The tool maintains several log files to track recent activity. The `output.log` file stores the latest console output from executed scripts, while `error_output.log` captures error messages and stack traces, making it easier to troubleshoot issues. Another log, `python_file_changes.log`, keeps a list of Python files that are created, modified, or executed within your working directories. By collecting this data, Code Watchdog can provide valuable insights and suggest fixes based on the stored information whenever needed.

Installing Code Watchdog

Installing Code Watchdog on your system is straightforward. It works on both Mac and Linux, and you can set it up using a single command. Cloning the repository, setting up the Conda environment, and installing the required dependencies like psutil, watchdog, Weave, and Anthropic are all handled by running:

```
git clone https://github.com/bdytx5/code_watchdog.git && cd code_watchdog && sh setup.sh
```

The installation also configures the logging infrastructure and monitoring scripts automatically. Part of the setup involves adding a bash alias for the `fix.py` script, making it easy to invoke the AI-powered debugging process.

Besides the installation command, the only commands you should need to run are activating the conda environment named "code_watchdog" with the command `conda activate code_watchdog`, and exporting your Anthropic API key with the command export `ANTHROPIC_API_KEY="your_api_key"`. My goal was to make the installation as simple as possible, and if you have any issues installing it, feel free to file an issue on the GitHub repo, and I'll fix the issue right away.

Once the installation is complete, and you have activated the code_watchdog environment, and set your Anthropic API key, you can now use code watchdog in your terminal with the `cw` command (I took inspiration from the famous cd command). This provides a quick and convenient way to trigger the tool and generate AI-powered fixes whenever you encounter issues in your code.

The tool tracks file activities such as creation, modification, and execution. These logs are prioritized so that the most recent activities—whether a file is created, modified, or executed—are placed at the top of the queue to be used as context, ensuring they are included first when context is needed.

Code Watchdog also captures two types of console outputs: standard output and error output. Standard output, including print statements and program results, is stored in output.log, making it easy to see what your code generated during its last execution. Errors, exceptions, and stack traces are recorded in error_output.log, providing a clear view of any runtime issues without requiring you to manually copy error messages from the terminal.

The Core logic

The core logic of Code Watchdog lies in the `fix.py` script, which uses Anthropic's Claude 3.5 API and Weave to generate and analyze AI-powered fixes based on logs, context, and

recent code changes. This script serves as the brains of the tool, providing actionable suggestions for errors based on the collected data. Fix.py is ran every time you use the 'cw' command.

The `fix.py` script accepts several command-line arguments to specify its focus. The first argument `<n>` determines how many recent modified or executed files should be analyzed. An optional second argument allows you to pass specific instructions to the AI, such as "Refactor the code to avoid division by zero." Additionally, the script can take a third argument to specify whether it should focus on error logs or console output by using the keywords `err` or `console`. For example, the command `cw 5 "Fix the bug" err` will analyze the last five modified files, use only the error log file, and generate a solution based on the provided instruction.

Here's the core logic script which powers Code Watchdog, which is ran evertime the `cw` command is invoked:

```
import anthropic
import sys
from pathlib import Path
import os
import re
import subprocess
import weave; weave.init("cw")
import os
import anthropic

client = anthropic.Client(
    api_key=os.getenv("ANTHROPIC_API_KEY")
)

# Define the log file paths
log_dir = os.path.expanduser("~/.cw")
log_file_path = os.path.join(log_dir, "output.log")
error_log_path = os.path.join(log_dir, "error_output.log")
monitor_log = Path("~/.cw/python_file_changes.log").expanduser()
solution_file_path = Path("~/.cw/solution.py").expanduser()  # Path to the solution file

def get_last_n_lines(file_path, n=40):
    """Read the last n lines of a file."""
    try:
        with open(file_path, 'r') as f:
            return ''.join(f.readlines()[-n:])
    except Exception as e:
        print(f"Error reading {file_path}: {e}")
```

```
        return ""

def get_unique_files_from_log(n):
    """Extract the last n unique modified Python files from the monitor log."""
    if not monitor_log.exists():
        print(f"Log file not found: {monitor_log}")
        sys.exit(1)

    unique_files = []
    seen_files = set()

    with open(monitor_log, 'r') as f:
        lines = f.readlines()

    # Iterate over the log lines in reverse to get the most recent first
    for line in reversed(lines):
        parts = line.split()
        if len(parts) > 1 and parts[0] in ["Modified:", "Created:", "Executed:"]:

            file_path = parts[1]


            if file_path not in seen_files and 'fix.py' not in file_path and 'file_monit
                seen_files.add(file_path)
                unique_files.append(file_path)

            if len(unique_files) >= n:
                break

    return unique_files

def read_file_contents(file_paths):
    """Read and return the contents of the specified files."""
    contents = ""
    for file_path in file_paths:
        try:
            with open(file_path, 'r') as f:
                contents += f"\n--- {file_path} ---\n{f.read()}"
        except Exception as e:
            print(f"Error reading {file_path}: {e}")
    return contents

@weave.op
def generate_fix_with_anthropic(error_log: str, recent_output: str, file_contents: str,
    """Query the Anthropic API with the error log, recent files content, and optional in
```

```
    # Prepare the user content for the message
    user_content = [
        {"type": "text", "text": f"I encountered the following error:\n\n{error_log}"},
        {"type": "text", "text": f"Here is the most recent console output:\n\n{recent_ou
        {"type": "text", "text": f"Here are the contents of some recent Python files:\n\
    ]

    if instruction:
        user_content.append({"type": "text", "text": f"Additional instruction: {instruct

    # Create the message using the Messages API
    message = client.messages.create(
        model="claude-3-5-sonnet-latest",
        max_tokens=1024,
        temperature=0,
        system="You are an expert Python programmer. Help resolve code errors efficientl
        messages=[
            {
                "role": "user",
                "content": user_content
            }
        ]
    )

    return message.content[0].text  # Correct access to the response content



def parse_claude_output(output: str) -> str:
    """Parse Claude's output into Python code and comments."""
    code_lines = []
    comment_lines = []

    code_block_start = re.compile(r"```python")
    code_block_end = re.compile(r"```")

    inside_code_block = False

    for line in output.splitlines():
        if code_block_start.match(line.strip()):
            inside_code_block = True
            continue  # Skip the start marker
        elif code_block_end.match(line.strip()):
```

```
            inside_code_block = False
            continue  # Skip the end marker

        if inside_code_block:
            code_lines.append(line)
        else:
            if line.strip():  # Avoid adding empty comment lines
                comment_lines.append(f"# {line}")

    if code_lines:
        return "\n".join(comment_lines + ["\n"] + code_lines)
    else:
        return output

def save_to_solution_file(content: str, file_path: Path):
    """Save the generated content to the solution.py file."""
    file_path.parent.mkdir(parents=True, exist_ok=True)  # Ensure directory exists
    with open(file_path, 'w') as f:
        f.write(content)
    print(f"Saved solution to {file_path}")

def open_file_in_vscode(file_path: Path):
    """Open the specified file in VSCode."""
    try:
        subprocess.run(["code", str(file_path)], check=True)
        print(f"Opened {file_path} in VSCode")
    except subprocess.CalledProcessError as e:
        print(f"Failed to open {file_path} in VSCode: {e}")
    except FileNotFoundError:
        print("VSCode executable 'code' not found. Make sure VSCode command line tools a




if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python read_last_modified.py <n> [<instruction>] [err|console]")
        sys.exit(1)

    try:
        n = int(sys.argv[1])
    except ValueError:
        print("Error: <n> must be an integer.")
        sys.exit(1)
```

```
    # Optional instruction parameter
    instruction = ""
    log_type = ""

    # Determine if the second argument is instruction or log type
    if len(sys.argv) > 2:
        if sys.argv[2] in ["err", "console"]:
            log_type = sys.argv[2]
        else:
            instruction = sys.argv[2]

    # Check for log_type in the third argument if not already set
    if not log_type and len(sys.argv) > 3:
        log_type = sys.argv[3]

    # Get the last n unique modified files
    unique_files = get_unique_files_from_log(n)
    print(unique_files)
    file_contents = read_file_contents(unique_files) if unique_files else ""

    # Read the last 40 lines of output.log and error_output.log
    recent_output = get_last_n_lines(log_file_path, 40)
    error_log = get_last_n_lines(error_log_path, 40)

    if error_log.strip() or recent_output.strip():
        print("\n--- Generating Fix with Anthropic ---")
        if log_type == "err":
            fix = generate_fix_with_anthropic(error_log, "", file_contents, instruction)
        elif log_type == "console":
            fix = generate_fix_with_anthropic("", recent_output, file_contents, instruct
        else:
            fix = generate_fix_with_anthropic(error_log, recent_output, file_contents, i

        print(f"\n--- Suggested Fix ---\n{fix}")

        # Parse the fix and check if it contains Python code
        parsed_content = parse_claude_output(fix)

        if "```python" in fix:
            save_to_solution_file(parsed_content, solution_file_path)
            open_file_in_vscode(solution_file_path)
        else:
            print("\n--- Solution ---")
            print(parsed_content)
```

```
    else:
        print("\nNo recent output or errors found.")
```

The script collects context from logs and recent file changes to ensure relevant context to generate accurate solutions. It reads from `python_file_changes.log` to track the most recent changes or executions of Python files, and it pulls information from `output.log` and `error_output.log` to understand recent console outputs and errors. This combination of logs allows `fix.py` to gather all necessary context before generating a solution.

Once the relevant context is gathered, `fix.py` calls the Claude 3.5 API from Anthropic to analyze the information and generate a solution. It sends the recent output or errors, the contents of recently modified files, and any additional instructions provided by the user. The AI processes this data and returns a suggested fix, which could range from code snippets to detailed explanations for addressing the issue.

Here's where we call the API:

```
@weave.op
def generate_fix_with_anthropic(error_log: str, recent_output: str, file_contents: str,
    """Query the Anthropic API with the error log, recent files content, and optional in


    # Prepare the user content for the message
    user_content = [
        {"type": "text", "text": f"I encountered the following error:\n\n{error_log}"},
        {"type": "text", "text": f"Here is the most recent console output:\n\n{recent_ou
        {"type": "text", "text": f"Here are the contents of some recent Python files:\n\
    ]


    if instruction:
        user_content.append({"type": "text", "text": f"Additional instruction: {instruct


    # Create the message using the Messages API
    message = client.messages.create(
        model="claude-3-5-sonnet-latest",
        max_tokens=1024,
        temperature=0,
        system="You are an expert Python programmer. Help resolve code errors efficientl
        messages=[
            {
```

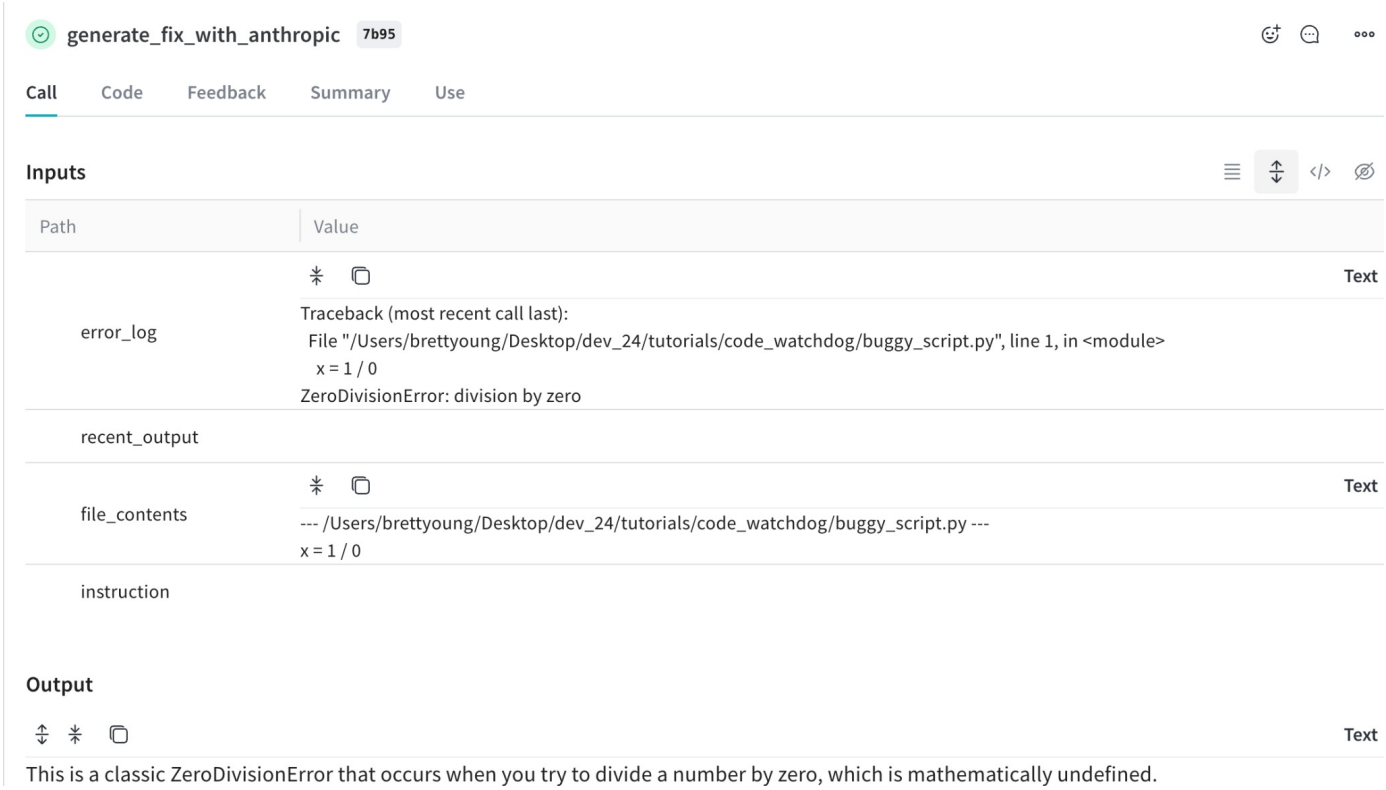
```
                "role": "user",
                "content": user_content
            }
        ]
    )


    return message.content[0].text  # Correct access to the response content
```

Weave's role is focused on visualizing and tracking context data throughout the development process. I was even able to take advantage of Weave during the initial phases of building Code Watchdog, **as missing or incomplete context files were a recurring issue that complicated troubleshooting**. Using Weave as a logging and visualization tool helped quickly detect missing files and ensured that the AI received accurate context. It also provided insight into the sequence of file changes and errors, making it easier to debug the logging infrastructure. By adding the `@weave.op` decorator, we can automatically visualize calls to the model inside Weave.

After using the tool, we can navigate to Weave and visualize the exact inputs that were given to the model, and monitor how well the model is performing. For example, here's a Weave trace from a Code Watchdog command I ran for a simple coding error. First I ran the following command:

`cw 1 err`

Then, after navigating to Weave, we can see the following logs:



The error is in the file `buggy_script.py` on line 1:
```python
x = 1 / 0  # This causes a ZeroDivisionError
```

Additionally, Code Watchdog launches the solution file locally in VSCode as shown here:

Understanding Code Watchdog's CLI arguments

In order to use the `cw` command effectively, it's important to understand the arguments that can be passed. These include specifying the number of context files, providing an optional custom instruction, and selecting whether to focus on error logs or console output. Each argument plays a role in tailoring the AI's response based on the most relevant information.

The first argument determines how many recent files to analyze. The tool retrieves the specified number of the most recently created, modified, or executed files from the logs. Since recent activity is prioritized, any files you have run, edited, or created last will automatically be placed at the top of the queue, ensuring they are included in the analysis.

For example, running `cw 3` will analyze the last three files based on your recent interactions, while `cw 5` will expand the context to include five recent files. You can even run `cw 0` if you are just looking to debug a trivial pip error, or any other issue that doesn't require context files. This argument is also optional, and if not specified, will use a value of 3 as the number of recent context files to use.

The second argument allows you to pass an optional custom instruction to the AI, which helps tailor the generated solution. This is useful when you need to address a specific bug or optimize a section of code. For instance, running `cw 4 "Fix the division by zero error"` will analyze the last four files and instruct the AI to focus on that specific issue.

Similarly, `cw 3 "Refactor the code for better readability"` will ask the AI to generate suggestions for improving the readability of the code across the last three modified or executed files.

The third and final argument (also optional) lets you specify whether the tool should focus on error logs or console output. By default, the AI will analyze both `error_output.log` and `output.log` to provide a comprehensive solution based on previous errors and console output. However, if you prefer to narrow the focus of context to provide to the model, you can specify the type of log to use. For example, `cw 3 err` will instruct the AI to only consider the error logs when generating a fix, while `cw 2 console` will limit the analysis to the latest console output. This flexibility allows you to choose the most relevant data for each debugging session.

You can also combine these arguments to create a more precise command. For instance, `cw 3 "Resolve the IndexError" err` will analyze the last three files, focus on error logs, and provide a solution specific to the IndexError. Another example is `cw 2 "Optimize the loop logic" console`, which gathers the last two recent context files, analyzes console output, and asks the AI to suggest improvements for the loop logic.

These examples show how to use the `cw` command to align the AI-powered solutions with your specific needs. By adjusting the number of context files, adding custom instructions, and choosing which logs to analyze, you can ensure that the generated solutions are accurate and relevant.

Python Debugger demo: Using Code Watchdog for Claude-powered bug fixes

Now we will go through how to install Code Watchdog and use it to solve some coding bugs. Additionally, I'll show you how it integrates with Weave so we can track all our usage of Code Watchdog! ***Please note that Code Watchdog currently works only on Linux and Mac systems. Additionally, our file monitoring system monitors all files underneath your Desktop directory, so any scripts must reside in a directory nested within the Desktop of your system.***

Step 1: Ensure Conda and API keys are set up

Before proceeding, make sure you have Conda installed. If it isn't installed, you can follow the instructions at the Miniconda website to set it up.

Step 2: Install Code Watchdog

Once Conda is installed and your API key is set, open your terminal and run the following

command to install code_watchdog:

```
git clone https://github.com/bdytx5/code_watchdog.git && cd code_watchdog && sh setup.sh
```

This command will clone the repository, set up a new Conda environment, install all necessary dependencies (like psutil, weave, and anthropic), and configure the logging infrastructure. Once installed, activate the new environment by running:

```
conda activate code_watchdog
```

Additionally, you need to set your Anthropic API key. This key allows code_watchdog to interact with Anthropic's Claude API for generating solutions. Set your API key with the following command:

```
export ANTHROPIC_API_KEY="your-api-key-here"
```

Replace "`your-api-key-here`" with your actual API key. To avoid having to set the key every time, you can add the export command to your .bashrc or .zshrc configuration file.

Step 3: Create a buggy Python file

To see Code Watchdog in action, create a small Python script with an intentional error. This will allow us to trigger the tool and generate a solution. Use the following command to create the file:

```
echo "x = 1 / 0" > buggy_script.py
```

This file contains a ZeroDivisionError that we will use to test the tool.

Step 4: Run the buggy script

Run the script to trigger the error and allow Code Watchdog to log it:

```
python buggy_script.py
```

You should see the following error message in the terminal:

```
Traceback (most recent call last): File "buggy_script.py", line 1, in <module> x = 1 / 0 ZeroDivisionError: division by zero
```

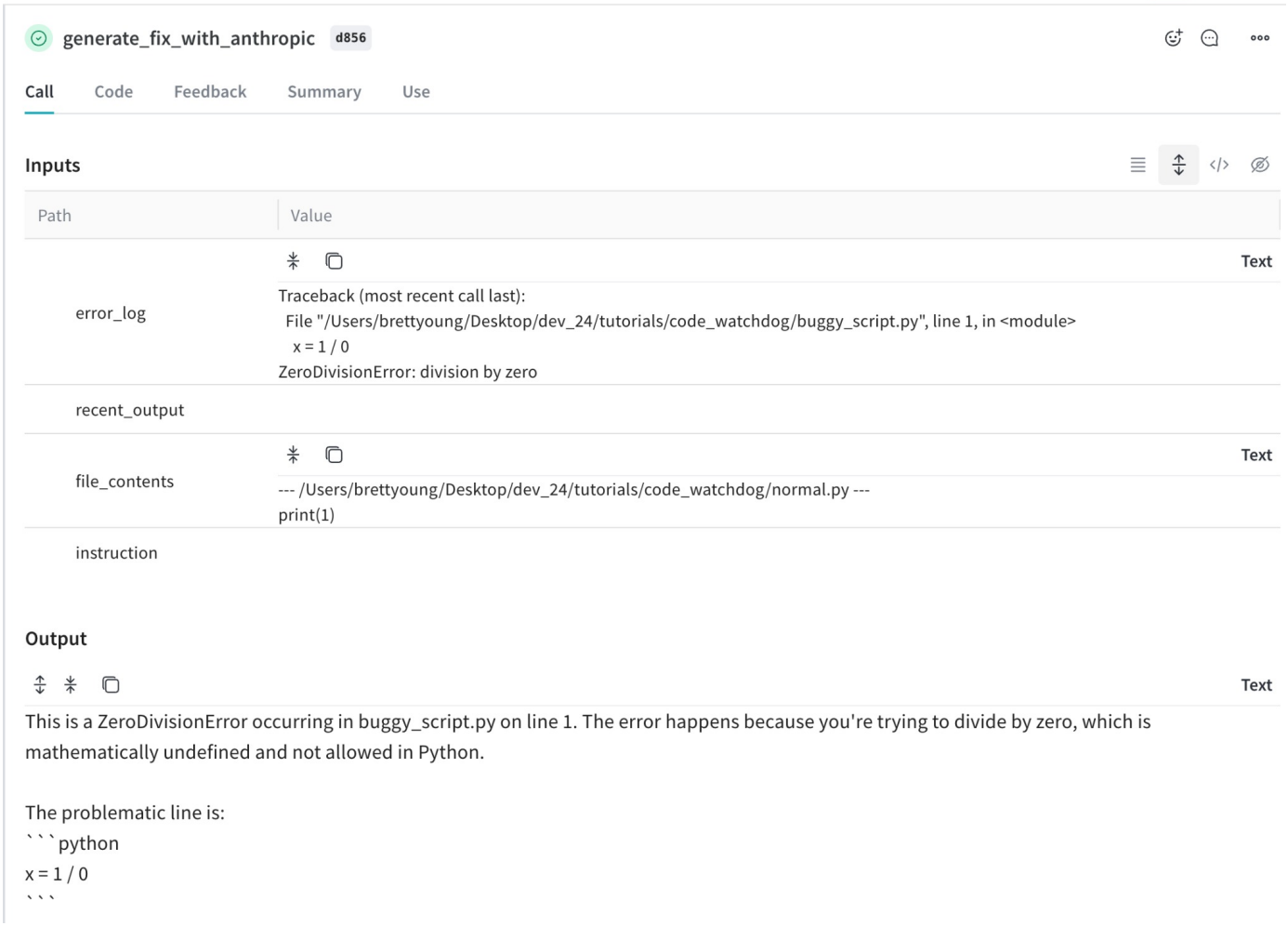
Step 5: Use Code Watchdog to generate a fix

With the error now logged, run the following `cw` command to analyze the issue and generate a solution:

```
cw 1 err
```

Notice that I didn't even provide an additional instruction. That's ok, Claude 3.5 Sonnet is smart enough to infer the issue, and provide me with a quick explanation to the issue. Overall, the command tells the tool to analyze the latest file (`buggy_script.py`), review only the latest error log, and provide a solution for the ZeroDivisionError.

Inside Weave, you can see our call to Code Watchdog:



Here, you see that the `recent_ouput` field is empty. That's because we used the "`err`" argument in the `cw` command, which only includes the most recent error log to the model. Using Weave, I was able to leverage this exact trace to quickly debug some errors in the Code Watchdog logging backend, which honestly saved me a ton of time. Before this project, I didn't realize Weave was useful for debugging, but it turned out to be really helpful.

The Future of Agents

Our tool offers a powerful way to debug code by combining AI-powered solutions with passive logging. By integrating tools like Anthropic's Claude 3.5 Sonnet and W&B Weave, it allows developers to troubleshoot issues efficiently without disrupting their workflow. The tool captures recent activities, analyzes errors and outputs, and provides targeted

solutions with just a quick command.

One cool feature Anthropic also unveiled is **computer control**, which allows models to interact with software interfaces like humans—clicking buttons, typing, and navigating programs. Though still experimental, this capability opens new possibilities for automation, enabling AI to actively engage with digital environments.

With some simple modifications, I think we could create our own version of computer control, focused on using Code Watchdog autonomously. By making model calls every time the context changes, the tool could decide whether to run a `cw` command and determine which arguments to use. **This would allow Code Watchdog to take proactive actions without waiting for user input. Adding feedback mechanisms could further refine the tool, allowing developers to guide its behavior and improve its decision-making over time.** *These enhancements could turn Code Watchdog into a fully autonomous assistant, monitoring and correcting code in real time, bringing us closer to the future of hands-free coding.*

Thanks for reading, and I hope you enjoyed learning about building an AI coding Agent. Stay tuned for future updates and new possibilities in AI-powered development. Feel free to check out the repo here.

Related Articles

Building a real-time answer engine with Llama 3.1 405B and W&B Weave

Infusing llama 3.1 405B with internet search capabilities!!

Vision fine-tuning GPT-4o on a custom dataset

Learn to vision fine-tune GPT-4o on a custom dataset, with evaluation and tracking.

Claude 3.5 Sonnet on Vertex AI: Python quickstart

Here's how to get up and running with the newest model from Anthropic

Training a KANFormer: KAN's Are All You Need?

We will dive into a new experimental architecture, replacing the MLP layers in transformers with KAN layers!

Add a comment

Tags: Articles